

MATH 391: Numerical Linear Algebra

Python Cheat Sheet for MATLAB Users

NumPy • SciPy • Matplotlib • Numerical Linear Algebra • Machine Learning

Key Idea

The central philosophy of numerical linear algebra is

Mathematical Problem $\longrightarrow$ Matrix Structure $\longrightarrow$ Numerical Algorithm $\longrightarrow$ Python Implementation

Do not merely ask, “Which Python command gives the answer?”

Ask instead:

What mathematical operation is Python actually performing?

1. The Basic Python Scientific Computing Stack

For most of MATH 391:

```
import numpy as np
import scipy.linalg as la
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
```

Later, for optimization and machine learning:

```
import torch
```

Package	Main Purpose	MATLAB Analogy
NumPy	arrays, matrix operations	core MATLAB
SciPy	numerical algorithms	MATLAB numerical routines
SciPy Sparse	sparse matrices	sparse(...)
Matplotlib	plotting	plot, surf, contour
PyTorch	tensors, gradients, ML	Deep Learning Toolbox

2. MATLAB $\longleftrightarrow$ NumPy

The Most Important Difference: Indexing

MATLAB starts counting at 1.

Python starts counting at 0.

MATLAB index i $\longleftrightarrow$ Python index $i - 1$

Operation	MATLAB	Python
First entry	<code>x(1)</code>	<code>x[0]</code>
Last entry	<code>x(end)</code>	<code>x[-1]</code>
Entry (i, j)	<code>A(i, j)</code>	<code>A[i, j]</code>
First row	<code>A(1, :)</code>	<code>A[0, :]</code>
First column	<code>A(:, 1)</code>	<code>A[:, 0]</code>
Last column	<code>A(:, end)</code>	<code>A[:, -1]</code>

Matrix Creation

MATLAB:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

```
A = np.array([[1, 2],
              [3, 4]])
```

Common constructors:

```
np.zeros((m, n))
np.ones((m, n))
np.eye(n)

np.arange(0, 10, 1)
np.linspace(0, 1, 100)

np.random.rand(m, n)
np.random.randn(m, n)
```

Dimensions:

```
A.shape
A.ndim
A.size

m, n = A.shape
```

3. Vectors: A Major MATLAB/Python Difference

MATLAB distinguishes row and column vectors naturally.

NumPy has a genuine one-dimensional array:

```
x = np.array([1, 2, 3])
print(x.shape)
# (3,)
```

This is neither explicitly 3×1 nor 1×3 .

Column vector:

```
x_col = np.array([[1],
                  [2],
                  [3]])
```

or

```
x_col = x.reshape(-1, 1)
```

Row vector:

```
x_row = x.reshape(1, -1)
```

Numerical Warning

For a 1-D NumPy array,

$$x^T = x$$

in the sense that `x.T` does not change its shape.

Thus

```
x.T
```

does *not* convert a 1-D array into a column or row vector.

4. Matrix Operations

Suppose $A, B \in \mathbb{R}^{n \times n}$.

Operation	Mathematics	Python
Addition	$A + B$	<code>A + B</code>
Subtraction	$A - B$	<code>A - B</code>
Matrix product	AB	<code>A @ B</code>
Elementwise product	$A \odot B$	<code>A * B</code>
Scalar product	αA	<code>alpha * A</code>
Transpose	A^T	<code>A.T</code>
Conjugate transpose	A^*	<code>A.conj().T</code>
Matrix power	A^k	<code>np.linalg.matrix_power(A,k)</code>
Elementwise power	a_{ij}^k	<code>A**k</code>
Trace	$\text{tr}(A)$	<code>np.trace(A)</code>
Determinant	$\det(A)$	<code>np.linalg.det(A)</code>
Rank	$\text{rank}(A)$	<code>np.linalg.matrix_rank(A)</code>

Numerical Warning

In MATLAB,

$$A * B$$

means matrix multiplication.

In NumPy,

```
A * B
```

means **elementwise multiplication**.

For matrix multiplication use

```
A @ B
```

This is probably the single most important MATLAB-to-Python habit to change.

5. Dot Products, Inner Products, and Outer Products

For real vectors,

$$x^T y = \sum_i x_i y_i.$$

Python:

```
x @ y
np.dot(x, y)
```

For complex vectors, the standard inner product uses conjugation:

$$\langle x, y \rangle = x^* y.$$

```
np.vdot(x, y)
```

Outer product:

$$xy^T.$$

```
np.outer(x, y)
```

6. Useful Matrix Structure

Diagonal:

```
D = np.diag([1, 2, 3])
d = np.diag(A)
```

Upper/lower triangular parts:

```
U = np.triu(A)
L = np.tril(A)
```

Symmetry check:

```
np.allclose(A, A.T)
```

Hermitian check:

```
np.allclose(A, A.conj().T)
```

7. Norms

Vector p -norm:

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}.$$

Important cases:

$$\|x\|_1 = \sum_i |x_i|,$$

$$\|x\|_2 = \sqrt{x^T x},$$

$$\|x\|_\infty = \max_i |x_i|.$$

Python:

```
np.linalg.norm(x, 1)
np.linalg.norm(x, 2)
np.linalg.norm(x, np.inf)
```

Matrix norms:

$$\|A\|_1 = \max_j \sum_i |a_{ij}|,$$

$$\|A\|_\infty = \max_i \sum_j |a_{ij}|,$$

$$\|A\|_2 = \sigma_{\max}(A).$$

Frobenius norm:

$$\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2}.$$

```
np.linalg.norm(A, 1)
np.linalg.norm(A, 2)
np.linalg.norm(A, np.inf)
np.linalg.norm(A, 'fro')
```

8. Linear Systems: $Ax = b$

The fundamental computational problem is

$$Ax = b.$$

MATLAB:

```
x = A\b
```

Python:

```
x = np.linalg.solve(A, b)
```

or

```
x = la.solve(A, b)
```

Numerical Warning

Do **not** normally compute

$$x = A^{-1}b$$

using an explicitly formed inverse.

Avoid:

```
x = np.linalg.inv(A) @ b
```

Prefer:

```
x = np.linalg.solve(A, b)
```

Numerical linear algebra is about solving the problem efficiently and stably, not blindly translating the symbolic formula $A^{-1}b$.

9. Residual and Error

Given an approximate solution $\hat{x}$,

$$r = b - A\hat{x}$$

is the **residual**.

Python:

```
r = b - A @ x
residual_norm = np.linalg.norm(r)
```

Relative residual:

$$\frac{\|b - A\hat{x}\|}{\|b\|}.$$

```
rel_res = np.linalg.norm(b - A @ x) / np.linalg.norm(b)
```

If the exact solution x is known,

$$e = x - \hat{x}$$

is the actual error.

Key Idea

Residual and error are **not the same thing**.

A tiny residual does not necessarily imply a tiny solution error if A is badly conditioned.

10. Condition Number

For a nonsingular matrix,

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

For the 2-norm,

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

Python:

```
kappa = np.linalg.cond(A)
```

Interpretation:

$$\kappa(A) \approx 1 \quad \Rightarrow \quad \text{well conditioned,}$$

while

$$\kappa(A) \gg 1 \quad \Rightarrow \quad \text{ill conditioned.}$$

Key Idea

Conditioning describes sensitivity of the *problem*.

Stability describes behavior of the *algorithm*.

These are different ideas.

11. Gaussian Elimination and LU Factorization

Gaussian elimination transforms

$$Ax = b$$

into an upper triangular system.

In matrix language, elimination leads to

$$A = LU$$

when no row permutation is required.

In practical computation, pivoting gives

$$PA = LU.$$

SciPy:

```
P, L, U = la.lu(A)
```

Numerical Warning

Be careful with permutation conventions.
 For `scipy.linalg.lu`, the returned matrices satisfy

$$A = PLU.$$

Thus the convention used by a software routine must always be checked rather than assumed from the mathematical notation.

For repeated systems

$$Ax = b_1, \quad Ax = b_2, \quad \dots,$$

factor A once.

```
lu, piv = la.lu_factor(A)
x1 = la.lu_solve((lu, piv), b1)
x2 = la.lu_solve((lu, piv), b2)
```

Key Idea

The expensive work is the factorization.
 Once

$$PA = LU$$

has been computed, new right-hand sides require only triangular solves.

12. Triangular Systems

For

$$Lx = b$$

use forward substitution.

For

$$Ux = b$$

use backward substitution.

SciPy:

```
x = la.solve_triangular(L, b, lower=True)
x = la.solve_triangular(U, b, lower=False)
```

13. Cholesky Factorization

If A is symmetric positive definite,

$$A = LL^T$$

or equivalently

$$A = R^T R.$$

Python:

```
L = np.linalg.cholesky(A)
```

Solve:

$$LL^T x = b.$$

```
y = la.solve_triangular(L, b, lower=True)
x = la.solve_triangular(L.T, y, lower=False)
```

Key Idea

When A is symmetric positive definite, exploit that structure. Cholesky is preferable to treating A as an arbitrary matrix.

14. Iterative Methods

Instead of obtaining x in one factorization-based computation, iterative methods generate

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

with the hope that

$$x^{(k)} \rightarrow x.$$

These methods are particularly important for very large sparse systems.

14.1 Jacobi Method

Split

$$A = D + L + U,$$

where D is diagonal.

Then

$$x^{(k+1)} = D^{-1} \left[b - (L + U)x^{(k)} \right]$$

Componentwise:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Python implementation without explicitly forming D^{-1} :

```
D = np.diag(A)
R = A - np.diag(D)

x_new = (b - R @ x) / D
```

14.2 Gauss-Seidel

With

$$A = (D + L) + U,$$

$$(D + L)x^{(k+1)} = b - Ux^{(k)}$$

Python:

```
DL = np.tril(A)
U = np.triu(A, 1)

x_new = la.solve_triangular(
    DL,
    b - U @ x,
    lower=True
)
```

14.3 Convergence

For a stationary iteration

$$x^{(k+1)} = Tx^{(k)} + c,$$

convergence for every initial guess occurs when

$$\rho(T) < 1,$$

where

$$\rho(T) = \max_i |\lambda_i(T)|$$

is the spectral radius.

15. Sparse Matrices

Large numerical problems often contain mostly zeros.

Instead of storing every entry, use sparse storage.

```
A_sparse = sp.csr_matrix(A)
```

Useful formats:

Format	Typical Purpose
CSR	matrix-vector products, row operations
CSC	column operations, many sparse solvers
COO	convenient sparse matrix construction

Sparse solve:

```
x = spla.spsolve(A_sparse, b)
```

Key Idea

For a huge sparse matrix, never convert to dense form merely because the dense syntax looks familiar.

16. Conjugate Gradient

CG solves

$$Ax = b$$

for symmetric positive-definite A .

```
x, info = spla.cg(A_sparse, b)
```

Typical interpretation:

```
if info == 0:
    print("CG converged")
else:
    print("CG did not fully converge")
```

CG searches over Krylov subspaces

$$\mathcal{K}_k(A, r_0) = \text{span} \{r_0, Ar_0, A^2r_0, \dots, A^{k-1}r_0\}.$$

17. Eigenvalues and Eigenvectors

An eigenpair satisfies

$$Av = \lambda v, \quad v \neq 0.$$

General matrix:

```
eigvals, eigvecs = np.linalg.eig(A)
```

Here,

```
eigvals[i]
```

corresponds to the eigenvector

```
eigvecs[:, i]
```

For symmetric/Hermitian matrices:

```
eigvals, eigvecs = np.linalg.eigh(A)
```

Python / MATLAB Tip

If A is symmetric, prefer `eigh` rather than `eig`.
Numerical algorithms should exploit known matrix structure.

18. Power Iteration

Power iteration estimates the dominant eigenvector.

Start with

$$v_0 \neq 0.$$

Then repeatedly compute

$$w_k = Av_k,$$

$$v_{k+1} = \frac{w_k}{\|w_k\|_2}.$$

Python:

```
v = np.random.randn(A.shape[0])
v = v / np.linalg.norm(v)

for k in range(100):
    v = A @ v
    v = v / np.linalg.norm(v)
```

Estimate the eigenvalue using the Rayleigh quotient:

$$\rho(v) = \frac{v^T A v}{v^T v}$$

```
lam = (v @ (A @ v)) / (v @ v)
```

If v is normalized,

$$\lambda \approx v^T A v.$$

19. QR Factorization

For

$$A \in \mathbb{R}^{m \times n},$$

QR factorization writes

$$A = QR,$$

where

$$Q^T Q = I$$

and R is upper triangular.

Python:

```
Q, R = np.linalg.qr(A)
```

Reduced QR:

```
Q, R = np.linalg.qr(A, mode='reduced')
```

Check:

```
np.allclose(Q.T @ Q, np.eye(Q.shape[1]))
np.allclose(Q @ R, A)
```

20. Least Squares

For an overdetermined system

$$Ax \approx b,$$

we solve

$$\min_x \|Ax - b\|_2.$$

The normal equations are

$$A^T A x = A^T b.$$

If $A = QR$, then

$$Rx = Q^T b.$$

Python's direct least-squares routine:

```
x, residuals, rank, s = np.linalg.lstsq(
    A, b, rcond=None
)
```

Numerical Warning

Although the normal equations are mathematically valid,

$$A^T A x = A^T b,$$

explicitly forming $A^T A$ can worsen conditioning:

$$\kappa_2(A^T A) = \kappa_2(A)^2.$$

For serious numerical work, QR or SVD is generally preferable.

21. Singular Value Decomposition

Every matrix

$$A \in \mathbb{R}^{m \times n}$$

has an SVD

$$A = U \Sigma V^T.$$

Python:

```
U, s, Vt = np.linalg.svd(
    A,
    full_matrices=False
)
```

Important:

s

is a **1-D array of singular values**, not the matrix Σ .

To construct Σ :

```
Sigma = np.diag(s)
```

Reconstruction:

```
A_reconstructed = U @ np.diag(s) @ Vt
```

Singular values satisfy

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0.$$

They are related to eigenvalues by

$$A^T A v_i = \sigma_i^2 v_i.$$

Thus

$$\sigma_i = \sqrt{\lambda_i(A^T A)}.$$

22. Rank- k Approximation

The truncated SVD gives

$$A_k = U_k \Sigma_k V_k^T.$$

Python:

```
k = 2

A_k = (
    U[:, :k]
    @ np.diag(s[:k])
    @ Vt[:k, :]
)
```

The Eckart–Young theorem tells us that this is the best rank- k approximation in the 2-norm and Frobenius norm.

$$\|A - A_k\|_2 = \sigma_{k+1}.$$

This is fundamental in

compression, denoising, PCA, dimensionality reduction.

23. Pseudoinverse

If

$$A = U \Sigma V^T,$$

then

$$A^\dagger = V \Sigma^\dagger U^T.$$

Python:

```
A_pinv = np.linalg.pinv(A)
```

Least-squares/minimum-norm solution:

$$x = A^\dagger b.$$

24. Principal Component Analysis (PCA)

Suppose the rows of

$$X \in \mathbb{R}^{m \times n}$$

are observations and the columns are features.

First center the data:

$$X_c = X - \text{column means}.$$

Python:

```
Xc = X - np.mean(X, axis=0)
```

Compute the SVD:

$$X_c = U\Sigma V^T.$$

```
U, s, Vt = np.linalg.svd(
    Xc,
    full_matrices=False
)
```

Principal directions:

```
components = Vt
```

Projected coordinates:

```
scores = Xc @ Vt.T
```

For the first k components:

```
Vk = Vt[:k, :].T
Z = Xc @ Vk
```

Key Idea

PCA is not merely a mysterious machine-learning command.
At its core,

PCA is linear algebra—especially SVD.

It finds directions along which centered data exhibits large variation.

25. Covariance Matrix

For centered data X_c with m observations,

$$C = \frac{1}{m-1} X_c^T X_c.$$

Python:

```
C = Xc.T @ Xc / (Xc.shape[0] - 1)
```

or

```
C = np.cov(X, rowvar=False)
```

Because

$$X_c = U\Sigma V^T,$$

we obtain

$$C = V \frac{\Sigma^2}{m-1} V^T.$$

Therefore PCA can be understood through either:

eigendecomposition of covariance

or

SVD of centered data.

26. Distance Matrices

Euclidean distance:

$$d(x, y) = \|x - y\|_2.$$

Python:

```
d = np.linalg.norm(x - y)
```

Pairwise distances:

```
from scipy.spatial.distance import cdist

D = cdist(X, X)
```

The matrix D contains

$$D_{ij} = \|x_i - x_j\|_2.$$

Distance matrices appear naturally in clustering and data analysis.

27. Optimization

We seek

$$x^* = \arg \min_x f(x).$$

At a differentiable unconstrained minimum,

$$\nabla f(x^*) = 0.$$

The gradient points in the direction of greatest local increase.

Therefore

$$-\nabla f(x)$$

points in the direction of steepest local decrease.

28. Gradient Descent

Gradient descent:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

where $\alpha_k > 0$ is the learning rate or step size.

Example:

$$f(x) = \frac{1}{2} \|Ax - b\|_2^2.$$

Then

$$\nabla f(x) = A^T(Ax - b).$$

Python:

```
x = np.zeros(A.shape[1])
alpha = 0.01

for k in range(1000):

    grad = A.T @ (A @ x - b)

    x = x - alpha * grad
```

Key Idea

Least squares connects numerical linear algebra and machine learning:

$$\min_x \|Ax - b\|_2^2$$

can be attacked using QR, SVD, normal equations, or iterative optimization such as gradient descent.

29. Stochastic Gradient Descent

Suppose

$$f(w) = \frac{1}{N} \sum_{i=1}^N \ell_i(w).$$

Full gradient descent uses all N samples.

SGD uses one sample or a small batch:

$$w_{k+1} = w_k - \alpha_k \nabla \ell_{i_k}(w_k).$$

The gradient is cheaper but noisy.

30. PyTorch Automatic Differentiation

Example:

$$f(w_1, w_2) = w_1^2 + w_2^2.$$

```
import torch

w = torch.tensor(
    [1.0, 2.0],
    requires_grad=True
)

loss = w[0]**2 + w[1]**2

loss.backward()

print(w.grad)
```

Since

$$\nabla f = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix},$$

PyTorch automatically computes this derivative.

Optimization:

```
optimizer = torch.optim.SGD(
    [w],
    lr=0.01
)

optimizer.zero_grad()
loss = w[0]**2 + w[1]**2
loss.backward()
optimizer.step()
```

The conceptual sequence is

forward computation $\rightarrow$ loss $\rightarrow$ backpropagation $\rightarrow$ gradient $\rightarrow$ parameter update.

31. Artificial Neuron

A basic artificial neuron computes

$$z = w^T x + b$$

followed by an activation function

$$y = \phi(w^T x + b).$$

For sigmoid activation,

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Python:

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

z = w @ x + b
y = sigmoid(z)
```

Thus even a basic neural network begins with familiar linear algebra:

dot product $\rightarrow$ bias $\rightarrow$ nonlinearity.

32. Binary Classification

For a linear classifier,

$$z = w^T x + b.$$

A simple decision rule is

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

Python:

```
z = X @ w + b
y_pred = (z >= 0).astype(int)
```

For logistic classification,

$$p(y = 1 \mid x) = \sigma(w^T x + b).$$

33. Clustering: Core Linear-Algebra View

For k -means, each point x_i is assigned to the nearest centroid.

$$c(i) = \arg \min_j \|x_i - \mu_j\|_2.$$

Then update

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i.$$

The algorithm alternates between

assign points $\longleftrightarrow$ recompute centroids.

The key mathematical ingredients are vectors, Euclidean norms, means, and distance matrices.

34. Plotting in Python: Basic Plots and Error Plots

Numerical linear algebra is not only about computing numbers. Plots are often used to understand

data convergence errors eigenvalue behavior optimization

The standard plotting package is Matplotlib:

```
import matplotlib.pyplot as plt
```

34.1 Basic 2D Plot

Suppose

$$y = \sin x, \quad 0 \leq x \leq 2\pi.$$

Python:

```
x = np.linspace(0, 2*np.pi, 200)
y = np.sin(x)

plt.plot(x, y)

plt.xlabel('x')
plt.ylabel('sin(x)')
plt.title('Sine Function')

plt.grid(True)
plt.show()
```

The basic structure is

```
plt.plot(x, y)
plt.show()
```

Everything else improves readability.

34.2 MATLAB vs. Python Plotting

Operation	MATLAB	Python
Basic plot	<code>plot(x,y)</code>	<code>plt.plot(x,y)</code>
Grid	<code>grid on</code>	<code>plt.grid(True)</code>
x label	<code>xlabel('x')</code>	<code>plt.xlabel('x')</code>
y label	<code>ylabel('y')</code>	<code>plt.ylabel('y')</code>
Title	<code>title('Plot')</code>	<code>plt.title('Plot')</code>
Legend	<code>legend(...)</code>	<code>plt.legend()</code>
Log- y	<code>semilogy(x,y)</code>	<code>plt.semilogy(x,y)</code>
Log-log	<code>loglog(x,y)</code>	<code>plt.loglog(x,y)</code>
Scatter	<code>scatter(x,y)</code>	<code>plt.scatter(x,y)</code>

34.3 Line Style, Markers, and Labels

Example:

```
plt.plot(
    x,
    y,
    'o-',
    label='Numerical Solution'
)

plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.legend()
plt.show()
```

Common marker/line combinations include

Code	Meaning
'-'	solid line
'--'	dashed line
':'	dotted line
'o'	circle markers
's'	square markers
'o-'	circles connected by solid lines
'o--'	circles connected by dashed lines

34.4 Plotting More Than One Curve

Suppose

$$y_1 = \sin x, \quad y_2 = \cos x.$$

```
x = np.linspace(0, 2*np.pi, 200)

y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, label='sin(x)')
plt.plot(x, y2, label='cos(x)')

plt.xlabel('x')
plt.ylabel('y')
```

```
plt.title('Two Functions')

plt.grid(True)
plt.legend()
plt.show()
```

34.5 Scatter Plot

Scatter plots are particularly important for data analysis, classification, PCA, and clustering.

```
x = np.random.randn(100)
y = np.random.randn(100)

plt.scatter(x, y)

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Data Points')

plt.grid(True)
plt.show()
```

34.6 Plotting Numerical Error

Suppose the exact solution is known:

$$x_{\text{exact}}.$$

If the numerical approximation is

$$x_{\text{num}},$$

then the error may be measured by

$$e = \|x_{\text{exact}} - x_{\text{num}}\|.$$

Python:

```
error = np.linalg.norm(x_exact - x_num)
```

If an iterative method produces approximations

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots,$$

we may store

$$e_k = \|x - x^{(k)}\|_2.$$

For example:

```
errors = []

for k in range(maxiter):

    # update x here

    err = np.linalg.norm(x_exact - x)

    errors.append(err)
```

Then plot error against iteration number:

```

iterations = np.arange(len(errors))

plt.plot(iterations, errors, 'o-')

plt.xlabel('Iteration')
plt.ylabel('Error')
plt.title('Error vs Iteration')

plt.grid(True)
plt.show()

```

34.7 Semilog Error Plot

Errors often decrease by several orders of magnitude.

A normal plot may hide this behavior.

Instead use

```

plt.semilogy(iterations, errors, 'o-')

plt.xlabel('Iteration')
plt.ylabel('Error')
plt.title('Convergence History')

plt.grid(True)
plt.show()

```

This means

x -axis: linear scale, y -axis: logarithmic scale.

Key Idea

For iterative algorithms such as Jacobi, Gauss–Seidel, Conjugate Gradient, power iteration, and gradient descent, a semilog plot is often much more informative than an ordinary plot.

34.8 Residual Plot

For

$$Ax = b,$$

the residual at iteration k is

$$r_k = b - Ax_k.$$

Its norm is

$$\|r_k\|_2 = \|b - Ax_k\|_2.$$

A convergence history can therefore be generated by

```

residuals = []

for k in range(maxiter):

    # update x here

    r = b - A @ x

```

```
residuals.append(
    np.linalg.norm(r)
)
```

Plot:

```
plt.semilogy(residuals)

plt.xlabel('Iteration')
plt.ylabel('Residual Norm')

plt.title('Residual Convergence')
plt.grid(True)

plt.show()
```

Numerical Warning

Do not confuse

error with residual.

Error:

$$e_k = x - x_k.$$

Residual:

$$r_k = b - Ax_k.$$

Usually the exact solution x is unknown, so in practical computations the residual is much easier to calculate.

34.9 Relative Error and Relative Residual

Relative error:

$$\frac{\|x - x_k\|_2}{\|x\|_2}$$

Python:

```
relative_error = (
    np.linalg.norm(x_exact - x)
    / np.linalg.norm(x_exact)
)
```

Relative residual:

$$\frac{\|b - Ax_k\|_2}{\|b\|_2}$$

Python:

```
relative_residual = (
    np.linalg.norm(b - A @ x)
    / np.linalg.norm(b)
)
```

34.10 Log–Log Error Plots

Log–log plots are useful when studying how an error behaves as some numerical parameter h decreases.

Suppose

$$E(h) \approx Ch^p.$$

Taking logarithms gives

$$\log E \approx \log C + p \log h.$$

Therefore, on a log–log plot,

slope $\approx p$.

For example, suppose a method has second-order error:

$$E(h) = O(h^2).$$

Python:

```
h = np.logspace(-4, -1, 10)

error = h**2

plt.loglog(
    h,
    error,
    'o-',
    label='Numerical Error'
)

plt.loglog(
    h,
    h**2,
    '--',
    label='O(h^2)'
)

plt.xlabel('h')
plt.ylabel('Error')

plt.title('Error Convergence')

plt.grid(True, which='both')
plt.legend()

plt.show()
```

The reference line

$$h^2$$

allows us to visually check whether

$$E(h) \sim O(h^2).$$

34.11 Estimating the Observed Convergence Rate

If

$$E(h) \approx Ch^p,$$

then using two successive measurements,

$$\frac{E_2}{E_1} \approx \left(\frac{h_2}{h_1} \right)^p.$$

Therefore

$$p = \frac{\log(E_2/E_1)}{\log(h_2/h_1)}.$$

Python:

```
p = (
    np.log(error[1:] / error[:-1])
    /
    np.log(h[1:] / h[:-1])
)

print(p)
```

If the method is second order, we expect approximately

$$p \approx 2.$$

34.12 Plotting Eigenvalues in the Complex Plane

For eigenvalues

$$\lambda_j = \operatorname{Re}(\lambda_j) + i \operatorname{Im}(\lambda_j),$$

we can plot them in the complex plane.

```
lam = np.linalg.eigvals(A)

plt.scatter(
    lam.real,
    lam.imag
)

plt.axhline(0)
plt.axvline(0)

plt.xlabel('Real Part')
plt.ylabel('Imaginary Part')

plt.title('Eigenvalues')

plt.grid(True)
plt.show()
```

This is useful for visualizing spectral behavior.

34.13 Plotting Singular Values

Singular values often reveal numerical rank and low-rank structure.

```
s = np.linalg.svd(
    A,
    compute_uv=False
)

plt.semilogy(
    np.arange(1, len(s)+1),
    s,
```

```

    'o-')
)

plt.xlabel('Index')
plt.ylabel('Singular Value')

plt.title('Singular Value Decay')

plt.grid(True)
plt.show()

```

Rapid decay suggests that A may admit a good low-rank approximation.

34.14 Plotting Optimization Loss

For gradient descent or SGD, store

$$f(x_k)$$

during every iteration.

```

loss_history = []

for k in range(maxiter):

    # compute/update x

    loss = 0.5 * np.linalg.norm(
        A @ x - b
    )**2

    loss_history.append(loss)

```

Then:

```

plt.semilogy(loss_history)

plt.xlabel('Iteration')
plt.ylabel('Loss')

plt.title('Optimization Convergence')
plt.grid(True)

plt.show()

```

34.15 The Four Plots to Remember

ordinary data	→	<code>plt.plot</code>
scattered data	→	<code>plt.scatter</code>
iteration error/residual	→	<code>plt.semilogy</code>
convergence order	→	<code>plt.loglog</code>

Key Idea

For numerical computation, a plot should answer a mathematical question.

Examples:

Is my iteration converging?	→	residual plot,
How fast is it converging?	→	semilog plot,
What is the observed order?	→	log–log plot,
Is the matrix approximately low rank?	→	singular-value plot.

Plotting is therefore part of numerical analysis, not merely decoration.

35. Machine Precision

Floating-point arithmetic is finite precision.

Machine epsilon:

```
eps = np.finfo(float).eps
print(eps)
```

For double precision,

$$\epsilon_{\text{mach}} \approx 2.22 \times 10^{-16}.$$

Numerical Warning

Computer arithmetic is not exact real-number arithmetic.

In general,

$$\text{fl}(x \circ y) \neq x \circ y.$$

This is why conditioning, stability, scaling, and algorithm selection matter.

36. Never Test Floating-Point Equality Naively

Avoid:

```
A == B
```

when checking results of numerical calculations.

Prefer:

```
np.allclose(A, B)
```

For scalars:

```
np.isclose(a, b)
```

37. MATLAB $\longleftrightarrow$ Python Master Table

Operation	MATLAB	NumPy/SciPy
Matrix	<code>A=[1 2;3 4]</code>	<code>np.array([[1,2],[3,4]])</code>
Identity	<code>eye(n)</code>	<code>np.eye(n)</code>
Zeros	<code>zeros(m,n)</code>	<code>np.zeros((m,n))</code>
Matrix multiply	<code>A*B</code>	<code>A @ B</code>
Elementwise multiply	<code>A.*B</code>	<code>A * B</code>
Transpose	<code>A'</code>	<code>A.T</code>
Solve $Ax = b$	<code>A\b</code>	<code>np.linalg.solve(A,b)</code>
Norm	<code>norm(x)</code>	<code>np.linalg.norm(x)</code>
Rank	<code>rank(A)</code>	<code>np.linalg.matrix_rank(A)</code>
Condition number	<code>cond(A)</code>	<code>np.linalg.cond(A)</code>
Eigenvalues	<code>eig(A)</code>	<code>np.linalg.eig(A)</code>
QR	<code>qr(A)</code>	<code>np.linalg.qr(A)</code>
SVD	<code>svd(A)</code>	<code>np.linalg.svd(A)</code>
Pseudoinverse	<code>pinv(A)</code>	<code>np.linalg.pinv(A)</code>
Cholesky	<code>chol(A)</code>	<code>np.linalg.cholesky(A)</code>
Least squares	<code>A\b</code>	<code>np.linalg.lstsq(A,b,rcond=None)</code>
Diagonal	<code>diag(A)</code>	<code>np.diag(A)</code>
Upper triangle	<code>triu(A)</code>	<code>np.triu(A)</code>
Lower triangle	<code>tril(A)</code>	<code>np.tril(A)</code>
Maximum	<code>max(x)</code>	<code>np.max(x)</code>
Sum	<code>sum(x)</code>	<code>np.sum(x)</code>
Mean	<code>mean(x)</code>	<code>np.mean(x)</code>
Size	<code>size(A)</code>	<code>A.shape</code>
Length	<code>length(x)</code>	<code>len(x)</code>
Plot	<code>plot(x,y)</code>	<code>plt.plot(x,y)</code>

38. Which Tool Should I Use?

$Ax = b$	$\rightarrow$	<code>solve</code>
A SPD	$\rightarrow$	Cholesky / CG
A huge and sparse	$\rightarrow$	sparse iterative method
$Ax \approx b$	$\rightarrow$	QR / SVD / lstsq
$Av = \lambda v$	$\rightarrow$	eigenvalue algorithms
$A = A^T$	$\rightarrow$	<code>eigh</code>
dominant eigenpair	$\rightarrow$	power iteration
low-rank structure	$\rightarrow$	SVD
dimensionality reduction	$\rightarrow$	PCA/SVD
$\min f(x)$	$\rightarrow$	gradient methods

39. The Numerical Linear Algebra Mindset

Before writing Python code, ask:

1. What is the mathematical problem?

2. What are the dimensions?
3. Is the matrix dense or sparse?
4. Is it symmetric?
5. Is it positive definite?
6. Is it triangular?
7. Is it rank deficient?
8. Is the problem well conditioned?
9. Can I exploit a factorization?
10. Should I use a direct or iterative method?
11. How will I verify the answer?

Verification should become automatic:

```
# Linear system
np.linalg.norm(A @ x - b)

# QR
np.linalg.norm(A - Q @ R)

# Orthogonality
np.linalg.norm(Q.T @ Q - np.eye(Q.shape[1]))

# SVD
np.linalg.norm(A - U @ np.diag(s) @ Vt)

# Eigenpair
np.linalg.norm(A @ v - lam * v)
```

Key Idea

The goal of this course is not to memorize Python commands.
The goal is to reach the point where you see a computational problem and naturally think

structure $\rightarrow$ algorithm $\rightarrow$ implementation $\rightarrow$ verification.

Python is simply the language in which we express that numerical thinking.

Five Commands Worth Memorizing Immediately

If you remember nothing else during the first few classes:

```
A @ x # Matrix-vector product

np.linalg.solve(A, b) # Solve Ax = b

np.linalg.norm(x) # Norm

np.linalg.eig(A) # Eigenvalues/eigenvectors

U, s, Vt = np.linalg.svd(A) # SVD
```

And remember the MATLAB-to-Python rule:

A * B in MATLAB $\longrightarrow$ A @ B in NumPy.

Mathematics first. Algorithm second. Code third.

Then verify everything.